

FRAMEWORK SPRING - INJEÇÃO DE DEPENDÊNCIAS II

No conceito de inversão de controle, o container assume a responsabilidade de instanciar objetos no código. Para isso, o programador deve indicar, por meio de anotações, quais classes serão instanciadas. A anotação mais genérica é `@Component`, que define uma classe como um componente, ou bean, do Spring. Ao iniciar, o Spring Boot ou o Spring MVC escaneia o projeto e cria automaticamente instâncias dessas classes, gerenciando-as como beans.

Embora existam outras anotações, como `@Repository`, `@Service` e `@Controller`, a diferença entre elas é apenas semântica. Por exemplo, `@Controller` é utilizada em classes que recebem requisições HTTP, enquanto `@Repository` é aplicada a classes relacionadas ao banco de dados. No entanto, todas realizam a mesma função no contexto da inversão de controle.

SPRING BOOT - INVERSÃO DE CONTROLE

```

Spring Boot

1 @Component
2 public class ExemploComponente {
3     public String digaOla() {
4         return "Olá, Spring Boot!";
5     }
6 }
7
8 @Service
9 public class ExemploServico {
10
11     @Autowired
12     private ExemploComponente exemploComponente;
13
14     public String executeServico() {
15         return exemploComponente.digaOla();
16     }
17 }
18
19 @Controller
20 public class ExemploController {
21
22     @Autowired
23     private ExemploServico exemploServico;
24
25     @GetMapping("/ola")
26     @ResponseBody
27     public String digaOla() {
28         return exemploServico.executeServico();
29     }
30 }

```

No código apresentado, há uma classe marcada com a anotação `@Component`, o que indica que essa classe será instanciada pelo container de inversão de controle do Spring. Da mesma forma, há um exemplo de serviço utilizando a anotação `@Service`. Não há diferença funcional entre essas anotações, já que ambas serão instanciadas e gerenciadas automaticamente pelo container.

Além disso, o serviço está sendo injetado em outra classe, e há também um `@Controller`, responsável por receber requisições HTTP. Embora cada anotação indique uma função específica – como serviços ou controle de requisições – todas servem ao mesmo propósito: permitir que o container gerencie as instâncias de forma automática.

SPRING BOOT – INJEÇÃO DE DEPENDÊNCIAS

```

1 interface MensagemServico {
2     String getMessage();
3 }
4
5 @Component
6 class MensagemServicoImpl1 implements MensagemServico {
7     public String getMessage() { return "Serviço 1"; }
8 }
9
10 @Component
11 class MensagemServicoImpl2 implements MensagemServico {
12     public String getMessage() { return "Serviço 2"; }
13 }
14
15 @Component
16 @Primary // Quando não especificado, Indica que é a primeira a ser injetada.
17 class MensagemServicoImpl3 implements MensagemServico {
18     public String getMessage() { return "Serviço Primário"; }
19 }
20
21 @Component
22 class ConsumidorServico {
23     @Autowired // Injeção de dependência automática pelo tipo.
24     private MensagemServico mensagemServico;
25
26     @Autowired
27     @Qualifier("mensagemServicoImpl1") // Diz qual bean injetar quando >1
28     private MensagemServico mensagemServicoQualificado;
29
30     @Resource(name = "mensagemServicoImpl2") //permite especificar o nome do bean.
31     private MensagemServico mensagemServicoResource;
32
33     @Inject // Similar ao @Autowired, porém parte do JSR-330.
34     private MensagemServico mensagemServicoInject;
35 }

```

No código apresentado, observa-se a implementação de injeção de dependências utilizando várias anotações do Spring. Inicialmente, é definida uma interface `MensagemServico`, que será implementada por diversas classes. No contexto do Java, uma classe que implementa uma interface precisa fornecer a implementação de seus métodos.

As classes `MensagemServicoImpl1`, `MensagemServicoImpl2` e `MensagemServicoImpl3` implementam essa interface, e cada uma é marcada com a anotação `@Component`, o que indica que elas serão gerenciadas pelo container de inversão de controle. A anotação `@Primary` está presente na classe `MensagemServicoImpl3`, definindo-a como a implementação preferida quando houver múltiplos candidatos para injeção.

Na classe `ConsumidorServico`, várias formas de injeção de dependências são utilizadas. O campo `mensagemServico` recebe a anotação `@Autowired`, o que permite a injeção automática do bean correspondente. Quando existem múltiplas implementações da interface `MensagemServico`, o Spring injetará a classe marcada com `@Primary`, no caso, `MensagemServicoImpl3`. Outra forma de especificar qual bean deve ser injetado é por meio da anotação `@Qualifier`, como observado no campo `mensagemServicoQualificado`, que define explicitamente a implementação `MensagemServicoImpl1` como a instância a ser injetada.

Também é possível utilizar a anotação `@Resource`, que permite indicar diretamente o nome do bean a ser injetado, sem a necessidade de `@Autowired`. No campo `mensagemServicoResource`, o bean `MensagemServicoImpl2` será injetado. Por fim, a anotação `@Inject`, proveniente da especificação JSR-330, funciona de maneira similar a `@Autowired` e foi utilizada para injetar a instância `MensagemServicoImpl3`, que também é marcada como `@Primary`.

Este exemplo demonstra diversas maneiras de realizar a injeção de dependências no Spring, cada uma adequada a diferentes cenários, proporcionando flexibilidade no gerenciamento de beans pelo container.



5m



DIRETO DO CONCURSO

02. (PR-4/UFRJ/UFRJ/2023)

A injeção de dependência é uma técnica de design usada para obter a inversão de controle. O Spring Framework oferece um recurso de injeção de dependência que permite aos objetos definir suas próprias dependências que o contêiner Spring posteriormente injeta nelas. Assinale a alternativa que NÃO faz parte dos recursos de injeção de dependência do Spring mais recente.

- a) `@autowired`
- b) `@ioc`
- c) `@qualifier`
- d) `@DependsOn`
- e) `@inject`



Nesta questão, é necessário identificar a anotação que não possui relação com a injeção de dependências.

- a) (@Autowired) é amplamente utilizada no Spring para injetar dependências automaticamente.
- b) Apresenta o termo @IoC, que não corresponde a uma anotação válida no Spring.
- c) (@Qualifier) é empregada para especificar qual bean deve ser injetado quando há mais de uma implementação disponível.
- d) (@DependsOn) define que um bean depende de outro para ser inicializado.
- e) (@Inject) é uma anotação que também pode ser usada para injeção de dependência, conforme a especificação JSR-330.

03. (CESPE/CEBRASPE/TRT – 8ª REGIÃO (PA E AP)/2022)

Assinale a opção que apresenta a anotação que define no framework Spring uma classe como pertencente à camada de persistência.

- a) @repository
- b) @component
- c) @service
- d) @transient
- e) @autowired



A anotação mencionada refere-se às usadas na inversão de controle, que definem quais classes serão instanciadas pelo contêiner. No contexto do Spring, temos anotações como @component, @service, @repository e @controller, cada uma com um propósito específico. Para a camada de persistência, a anotação correta é @repository, que designa a classe como responsável pela interação com a fonte de dados. Já @service define serviços de negócio, @transient é usada para campos que não devem ser persistidos, e @Autowired realiza a injeção de dependências.

04. (UFSCAR/UFSCAR/2023)

Spring Boot é um framework de desenvolvimento de aplicativos Java que simplifica consideravelmente o processo de criação, configuração e implantação de aplicativos baseados em Spring. Qual anotação do Spring Boot é usada para injetar um serviço em um controlador?

- a) @service



10m

- b) @injectable
- c) @qualifier
- d) @component
- e) @autowired



A anotação @component refere-se às anotações relacionadas à inversão de controle, que indicam quais classes serão instanciadas pelo container de inversão de controle. Existem várias anotações nesse contexto, como @component, @service, @repository e @controller. No caso específico da persistência, a anotação correta é @repository, que identifica classes que gerenciam a interação com o banco de dados. As anotações @service e @controller têm outros propósitos no gerenciamento das camadas de serviço e de controle, respectivamente. A anotação @transient é utilizada para indicar que um campo em uma classe de domínio não deve ser salvo no banco de dados, ou seja, a classe em questão é válida, mas aquele campo não deve ser persistido. Já a anotação @Autowired é utilizada para realizar a injeção de dependências, facilitando o gerenciamento de beans no Spring.

05. (FUNDATEC/PROCERGS/2023)

Sobre injeção de dependência, analise as assertivas abaixo e assinale a alternativa correta.

- I – Recomenda-se implementar a injeção de dependência quando uma classe é dependente de outra.
- II – A injeção de dependência não permite desacoplar uma classe de suas classes dependentes, mas facilita o entendimento da implementação.
- III – Uma das formas de implementação é pela criação de um pool de objetos, no qual objetos são retirados para serem injetados em uma classe.

- a) Todas estão corretas.
- b) Todas estão incorretas.
- c) Apenas I e II estão corretas.
- d) Apenas I e III estão corretas.
- e) Apenas II e III estão corretas.



I – A injeção de dependência é uma prática recomendada justamente para resolver o problema de acoplamento forte entre classes. Quando uma classe depende de outra, em vez de instanciá-la diretamente (usando new), a injeção de dependência delega essa responsabilidade ao container, desacoplando a classe dependente da classe fornecedora da dependência.

II – A principal finalidade da injeção de dependência é exatamente desacoplar classes dependentes, permitindo maior flexibilidade e reutilização de código.

III – No contexto da injeção de dependências, pode-se usar um “pool de objetos” para gerenciar a criação e reutilização de instâncias. Esse é um dos padrões de gestão de objetos que um container pode adotar, especialmente em cenários em que a criação de muitas instâncias geraria problemas de desempenho. O Spring, por exemplo, permite configurar beans como “singleton”, “prototype”, ou utilizar um pool de objetos em casos específicos.



15m

06. (FUNDATEC/PROCERGS/2023)

Qual é a diferença entre inversão de controle e injeção de dependências?

- a) A inversão de controle é um princípio e a injeção de dependências é uma técnica.
- b) A inversão de controle é uma técnica e a injeção de dependências é um princípio.
- c) Não há diferença, os termos são sinônimos.
- d) A inversão de controle é um conceito antigo e a injeção de dependências é um conceito moderno.
- e) A injeção de dependências é uma técnica específica da linguagem Java.



Os termos “inversão de controle” e “injeção de dependências” não são sinônimos, pois referem-se a conceitos distintos. Enquanto a inversão de controle é um conceito que abrange a delegação de responsabilidades para um contêiner, a injeção de dependências é uma técnica que implementa esse conceito, facilitando a instância e a disponibilidade de objetos.

Além disso, ambos os conceitos são contemporâneos e não se limitam a uma única linguagem de programação, podendo ser aplicados em diversas linguagens. A inversão de controle vai além de um simples princípio, permitindo a criação de contêineres que instanciam objetos de maneira prática e eficiente.

07. (CONSULPLAN/TSE/2012)

No contexto do framework Spring existem, basicamente, dois tipos de injeção de dependência, sendo que em um deles, a dependência é resolvida por meio de um construtor do objeto a receber o objeto dependente. Este tipo é conhecido por

- a) bean injection.
- b) setter injection.
- c) factory injection.
- d) constructor injection.



Os tipos de injeção de dependência podem ser realizados de duas maneiras: por construtor ou por método setter. A injeção que utiliza o construtor do objeto para receber a dependência é conhecida como “constructor injection”.

08. (CESPE/CEBRASPE/STJ/2015)

Acerca de frameworks para camada de apresentação e persistência, julgue o item seguinte. Na injeção de dependência via setter, se a dependência não estiver disponível, a classe será criada de forma consistente, dado que as dependências necessárias para o funcionamento do objeto poderão ser passadas ao instanciá-lo.


20m

Na injeção de dependência via setter, a classe indica quais dependências necessita, e o contêiner é responsável por injetá-las. No entanto, afirmar que a classe será criada de forma consistente se uma dependência não estiver disponível é impróprio. A ausência de uma dependência necessária resulta em um erro em tempo de execução, impossibilitando a criação adequada da classe. Isso está relacionado à gestão de dependências, pois, sem as dependências, não há como garantir a consistência do objeto.

GABARITO

02. b**03. a****04. e****05. d****06. a****07. d****08. E**

Este material foi elaborado pela equipe pedagógica do Gran Concursos, de acordo com a aula preparada e ministrada pelo professor Gunter Ribeiro Amorim.

A presente gravação tem como objetivo auxiliar no acompanhamento e na revisão do conteúdo ministrado na videoaula. Não recomendamos a substituição do estudo em vídeo pela leitura exclusiva deste material.
